

XENONnT Event Viewer

Design, Implementation & Optimization

May 2026

Contents

1	Overview	2
1.1	Background	2
1.2	Core Features	2
1.3	Tech Stack	2
2	Architecture	2
2.1	Layered Design	2
2.2	Data Flow	3
2.3	Data Types	3
2.4	NPZ Bundle Format	3
3	Key Implementation Details	3
3.1	Canvas Management: Solving Ghosting	3
3.1.1	Problem Evolution	3
3.2	Interaction System	4
3.2.1	Peak Click Hit Testing	4
3.2.2	Legend Toggle	4
3.2.3	Scroll Zoom	4
3.2.4	PMT ID Hover	4
3.3	Peak List Sorting	5
3.4	Keyboard Shortcuts	5
3.5	Export	5
3.6	PMT Hit Pattern	5
3.7	Font & Style Optimization	5
3.8	Waveform Line Widths	5
3.9	Waveform Colors	6
4	Data Exploration	6
4.1	DALI Probe	6
4.2	Midway3 Probe	6
4.3	Data Hierarchy	6
4.4	Real Data Extraction	6
4.5	Transfer Issues	6

5	Testing & Validation	7
5.1	Automated Test Suite	7
5.2	End-to-End Verification	7
6	PyInstaller Packaging	7
6.1	Build Flags	7
6.2	Version Injection	7
6.3	CI (GitHub Actions)	8
7	Complete Pitfall Log	8
8	Known Limitations	8
9	References	9

1 Overview

1.1 Background

XENONnT is a dark matter detection experiment at Gran Sasso, Italy. Researchers need an interactive tool to browse event waveforms and PMT hit patterns. EventViewer is a cross-platform desktop app that loads pre-extracted NPZ data bundles for offline viewing.

1.2 Core Features

- **Event browser:** left panel with S1/S2 area filters, event number search
- **Peak list:** sortable table (Type, Area PE, Width ns, Rise ns, Time ns)
- **Main peak markers:** bold Main S1/Main S2 in peak list
- **3-layer waveform:** Top PMT (blue #2196F3) + Bottom (green #4CAF50) + Total (red #F44336)
- **Interactive zoom:** click waveform peak region or table row for single-peak zoom
- **Legend toggle:** click legend text to show/hide waveform layers
- **Scroll zoom:** mouse wheel on waveform panels
- **Page zoom:** +/- buttons or Ctrl+scroll adjust DPI
- **PMT ID hover:** hover over PMT scatter dots shows channel ID
- **PDF/PNG export:** Ctrl+S saves to Desktop; toolbar save button
- **Run Selector:** dropdown auto-scans for bundled NPZ files
- **Multi-format:** NPZ (peak_basics) model and NPZ (real peaks) from DALI

1.3 Tech Stack

Python 3.9+, PySide6, Matplotlib, NumPy, optional Strax/Straxen, PyInstaller.

2 Architecture

2.1 Layered Design

```
run_app.py                # Entry, CLI args, auto-load
+-- event_viewer_app/      # Qt GUI (PySide6)
|   +-- main_window.py    # MainWindow, PeakListWidget, RunSelector
|   +-- event_browser.py  # Event list, search, filters
|   +-- event_canvas.py   # Canvas + mouse/keyboard events
|   +-- data_manager.py   # Data loading & caching
|   +-- qt_compat.py      # PySide6/PySide2 compat
+-- event_plotter/        # Plotting engine
|   +-- plotter.py        # Core plotting functions
|   +-- io.py             # Data I/O, PMT geometry
|   +-- style.py          # Colors, fonts, rcParams
+-- scripts/              # CLI + build tools
```

```

|   +-- extract_event_bundle.py # Strax -> NPZ extraction
|   +-- build_mac.sh           # macOS build script
+-- dali_probe/                # DALI data probes
|   +-- extract_peaks_bundle.py # Real peaks extraction
|   +-- report.md              # Probe findings
+-- debug_reports/             # Debug test suite
    +-- 00_SUMMARY.md through 04_ui_simulation_tests.txt

```

2.2 Data Flow

1. NPZ file → DataManager.open_npz_bundle()
2. Loads events[], peaks_list[], eac_list[], pmt_positions, to_pe
3. EventBrowser fills event list, auto-selects first event
4. User clicks event → MainWindow._on_event_selected() → plot_event_full()
5. Peak click → EventCanvas hit test → peak_clicked Signal → plot_peak_zoom()

2.3 Data Types

Source	Waveform	PMT Pattern	data_top	Origin
NPZ peak_basics	Model pulse (bi-exp)	EAC scaled	No	Midway3 processed
NPZ real peaks	Real data samples	Real area_per_channel	Yes	DALI
Strax (cutax)	Real data	Real area_per_channel	Yes	Online/offline

Table 1: Data source comparison

2.4 NPZ Bundle Format

Keys: events, peaks_list (object dtype), eac_list (optional), pmt_x/y/array/i, to_pe, run_id, waveform_source (new: “real_peaks” or “model”).

3 Key Implementation Details

3.1 Canvas Management: Solving Ghosting

3.1.1 Problem Evolution

1. **fig.clear()** reuse: Old content persisted between renders — ghosting.
2. **fig.clf()**: Better but edge artifacts remained during scroll.
3. **delaxes**: Remove axes individually. Ghosting persisted.
4. **QScrollArea**: Wrapped canvas, setWidgetResizable(False). Colored pixel artifacts, garbled display.
5. **Final: Full replacement**. Each clear() creates a new Figure + FigureCanvasQ-Tagg. Old canvas destroyed via deleteLater().

Listing 1: EventCanvas.clear() final implementation

```
def clear(self):
    if self._hover_annot:
        try: self._hover_annot.remove()
        except Exception: pass
        self._hover_annot = None
    old_canvas = self._canvas
    self._fig = Figure(figsize=(16, 10), facecolor="white", dpi=self._base_dpi)
    self._canvas = FigureCanvasQTAgg(self._fig)
    self._canvas.mpl_connect('button_press_event', self._on_click)
    self._canvas.mpl_connect('scroll_event', self._on_scroll)
    self._canvas.mpl_connect('motion_notify_event', self._on_hover)
    layout = self.layout()
    if layout:
        idx = layout.indexOf(old_canvas)
        if idx >= 0: layout.insertWidget(idx, self._canvas)
        else: layout.addWidget(self._canvas)
    old_canvas.setParent(None)
    old_canvas.deleteLater()
    if self._toolbar is not None:
        self._toolbar.canvas = self._canvas # Fix save button
    self._update_canvas_size()
```

3.2 Interaction System

3.2.1 Peak Click Hit Testing

Each peak stores its time region in `ax._peak_regions` during rendering. Originally used strict interval matching (`x_start <= x <= x_end`), but narrow peaks ($< 1\mu\text{s}$) are under 1 pixel wide on screen. Final approach: proximity search (500 μs tolerance) with interval matching as fallback.

3.2.2 Legend Toggle

Three-layer waveform legend text is clickable. Detected via `txt.contains(event)` in `_on_click`. Toggles `set_visible()` on the corresponding artist group.

3.2.3 Scroll Zoom

Mouse wheel on waveform axes adjusts x/y limits by factor 1.25 (zoom in) / 0.8 (zoom out), centered at cursor position.

3.2.4 PMT ID Hover

`motion_notify_event` checks `coll.contains(event)` for PMT scatter collections. On hit, shows yellow annotation with PMT channel ID.

3.3 Peak List Sorting

QTableWidget defaults to string sorting. Custom `NumericItem` class overrides `__lt__` for numeric comparison. Each row stores original peak index via `Qt.UserRole` for correct click mapping.

3.4 Keyboard Shortcuts

Direction keys consumed by `QListWidget/QTableWidget`. Solution: `QShortcut` with `WindowShortcut` context for Left/Right (event navigation) and Escape (return to overview).

3.5 Export

Evolved from `QFileDialog` (z-order issues in .app) to direct Desktop save with auto-incrementing filename. Toolbar save fixed by updating `self._toolbar.canvas` after canvas replacement. PDF backend added via `--hidden-import`.

3.6 PMT Hit Pattern

Circle radius auto-computed from PMT positions (was hardcoded 47.9cm, actual max 65.6cm). Colorbar fixed: `plt.colorbar()` replaced with `ax.figure.colorbar()` to prevent wrong-figure attachment. Marker size iterations: $50 \rightarrow 25 \rightarrow 12 \rightarrow 25 \rightarrow 60 \rightarrow 120$.

3.7 Font & Style Optimization

Iteration	Base Font	Legend Font	Axes Width
Initial	9pt	+0 (9pt)	1.5
Round 1	11pt	+3 (14pt)	2.0
Round 2	11pt	+6 (17pt)	2.0
Final	11pt	+14 (25pt)	2.0

Table 2: Font evolution

Legend frame removed entirely in final version. Title font increased from `_rsize+1` to `_rsize+3`, supitle to `_rsize+4`.

3.8 Waveform Line Widths

Component	Initial	Round 1	Final
Event Top/Bot	0.25	0.75	0.75
Event Total	0.35	1.05	1.05
Zoom Top/Bot	0.8	2.4	2.4
Zoom Total	1.5	4.5	4.5
Real data step	[0.45,0.45,0.8]	[1.35,1.35,2.4]	[1.35,1.35,2.4]

Table 3: Line width evolution (all 3x final)

3.9 Waveform Colors

Changed from Nature journal palette (violet/orange/gray) to vibrant Material Design colors: Top PMT=#2196F3 (Blue), Bottom=#4CAF50 (Green), Total=#F44336 (Red).

4 Data Exploration

4.1 DALI Probe

DALI (`ssh dali`) is a XENON data node. Findings:

- Raw data: only `raw_records_aqmon` (monitor), no TPC physics `raw_records`
- Processed data: peaks with real `data`, `data_top`, `area_per_channel` (runs 043864, 044116, etc.)
- peaks dtype: 19 fields including waveform data (200 samples, PE/sample)

4.2 Midway3 Probe

Midway3 (`ssh midway3`) is the main compute cluster:

- `/project/lgrandi/xenont/processed/`: 2193 runs with `peak_basics`
- `/project2/`: additional runs
- RunDB query: 73,925 runs, 0 with stored `raw_records/records` locally
- Run 023756: only `raw_records_aqmon`, no TPC `raw_records`
- 280 runs have peaklets (with `data`, `area_per_channel`)

4.3 Data Hierarchy

```
raw_records -> records -> peaklets -> peaks -> peak_basics
  (raw)       (ADC)    (per-PMT) (merged) (metadata)
```

Midway3 stores: `peak_basics` (lightweight metadata), `event_info`, EAC. DALI additionally stores: `peaklets`, `peaks` (with waveform samples).

4.4 Real Data Extraction

1. SSH to DALI
2. Source XENON environment: `/cvmfs/xenon.opensciencegrid.org/releases/nT/e17.2025.07`
3. Run `dali_probe/extract_peaks_bundle.py --run 043864 --n 200`
4. Output NPZ: 9.1MB, 200 events, 3566 peaks
5. Transfer via `rsync` (multiple retries needed)

4.5 Transfer Issues

SCP from Midway3 frequently corrupts files >5MB. Workaround: `rsync` with `resume`, multiple retries, smaller samples (5-50 events) for quick validation.

5 Testing & Validation

5.1 Automated Test Suite

Located at `debug_reports/`:

Report	Tests	Result	Focus
01_static_analysis	11 files	2 issues	Imports, paths, print stmts
02_data_pipeline	9	ALL PASS	Mode switch, cache, index
03_rendering_edges	8	ALL PASS	Empty peaks, memory, DPI
04_ui_simulation	8	ALL PASS	Sorting, UserRole, markers

Table 4: Debug reports

5.2 End-to-End Verification

- 023756 NPZ (model): 50 events, 69 peaks/event — all loadable
- 043864 real peaks (5ev): 5 events, real waveforms — verified
- 043864 real peaks (200ev): 200 events, 3566 peaks — verified
- Cross-switch: NPZ → real peaks → NPZ — cache cleared correctly
- 20 event switches: no rendering errors
- 50 peak zooms: no area mismatches
- 100 renders: no memory leak detected

6 PyInstaller Packaging

6.1 Build Flags

```
pyinstaller --onedir --windowed --name XENONnT-EventViewer \  
--add-data "scripts/output/events_run_023756.npz:." \  
--hidden-import matplotlib.backends.backend_pdf \  
--hidden-import matplotlib.backends.backend_agg \  
--collect-submodules numpy --collect-data matplotlib \  
--exclude-module tkinter --exclude-module PyQt5 \  
--exclude-module scipy --exclude-module pandas \  
--distpath dist --workpath /tmp/pyinstaller_mac run_app.py
```

6.2 Version Injection

```
/usr/libexec/PlistBuddy -c "Set_CFBundleShortVersionString_2.0.1" \  
"dist/XENONnT-EventViewer.app/Contents/Info.plist"
```


6.3 CI (GitHub Actions)

Triggered by `v*` tag push. Four jobs: Ubuntu Linux, Windows, macOS ARM64, macOS x64.

7 Complete Pitfall Log

1. **QScrollArea ghosting:** Tried `setWidgetResizable(True/False)`, `setFixedSize`, `repaint()`, `draw_idle()` vs `draw()`. Failed. Final: full Figure+Canvas replacement.
2. **Toolbar save broken:** Canvas replaced but toolbar still referenced old canvas. Fix: update `toolbar.canvas`.
3. **PDF backend missing:** PyInstaller excluded `backend_pdf`. Fix: add `--hidden-import`.
4. **Peak list string sort:** `NumericItem` class + `UserRole` for original indices.
5. **.app opens Terminal:** `--onedir --windowed` instead of `--onefile`.
6. **strax import error:** Removed `strax` from `hidden_imports`.
7. **PMT circle mismatch:** Auto-compute from PMT positions, not hardcoded 47.9cm.
8. **plt.colorbar wrong figure:** Use `ax.figure.colorbar()` everywhere.
9. **Inline comments break syntax:** Global `color` replace inserted comments mid-function-call. Manual cleanup required.
10. **DALI/Midway3 transfer:** SCP corrupts large files. `Rsync` + retries.
11. **3-layer PMT all zeros:** EAC loading was guarded by `strax-mode` check. Removed guard.
12. **Event waveform blank:** Build cache contained old source. Clean rebuild fixed it.
13. **Scroll wheel garbled:** Trackpad momentum scroll overwhelmed `draw_idle()`. Switched to `draw()`.
14. **Event/peak switch ghosting:** `clf()` + `draw()` leaves artifacts in Qt. Full canvas replacement.
15. **Left panel frozen:** `QSplitter stretchFactor(0,0) + maxWidth(320)`. Changed to `(0,1)` and `maxWidth(500)`.

8 Known Limitations

- Run 023756 has no TPC `raw_records`; waveforms are model-generated
- Real peaks data must be manually extracted from DALI and transferred
- Strax mode loads synchronously on main thread; large runs may freeze UI
- Windows/Linux require CI or native build; cannot cross-compile from macOS

- .app uses ad-hoc signing; Gatekeeper may warn on first launch
- Run Selector scan paths may not include external NPZ files in packaged app

9 References

- XENONnT Straxen docs: <https://straxen.readthedocs.io>
- PySide6: <https://doc.qt.io/qtforpython-6/>
- Matplotlib: <https://matplotlib.org/stable/>
- PyInstaller: <https://pyinstaller.org/en/stable/>